08-02 PubSub 시스템 분석

개요

Solana의 PubSub(Publish-Subscribe) 시스템은 클라이언트가 블록체인 상태 변화를 실시간으로 구독하고 알림을 받을 수 있는 WebSocket 기반의 이벤트 알림 시스템입니다. 이 시스템을 통해 계정 변화, 트랜잭션 상태, 슬롯 업데이트 등 다양한 블록체인 이벤트를 실시간으로 모니터링할 수 있습니다.

아키텍처 구조

핵심 컴포넌트

```
pub struct PubSubService {
thread_hdl: JoinHandle<()>,
}
```

```
pub struct RpcSolPubSubImpl {
config: PubSubConfig,
subscription_control: SubscriptionControl,
current_subscriptions: Arc<DashMap<SubscriptionId, SubscriptionToken>>,
}
```

```
pub struct RpcSubscriptions {
subscriptions_tracker: SubscriptionsTracker,
```

```
notification_sender: Arc<Mutex<Sender<TimestampedNotificationEntry>>>,

t_cleanup: Option<JoinHandle<()>>,

exit: Arc<AtomicBool>,

// ... 기타 필드들

}
```

시스템 계층 구조

1. **PubSubService**: WebSocket 서버 관리 및 연결 처리
2. **RpcSolPubSubImpl**: 구독/구독 해제 RPC 메서드 구현
3. **RpcSubscriptions**: 구독 관리 및 알림 배포 엔진
4. **SubscriptionsTracker**: 구독 상태 추적 및 관리

구독 유형 및 인터페이스

지원되는 구독 유형

1. 계정 구독 (Account Subscription)

```
#[rpc(name = "accountSubscribe")]

fn  account_subscribe(

&self,

pubkey_str: String,

config: Option<RpcAccountInfoConfig>,

) -> Result<SubscriptionId>;
```

```
pub struct AccountSubscriptionParams {
pub pubkey: Pubkey,
pub commitment: CommitmentConfig,
pub data_slice: Option<UiDataSliceConfig>,
pub encoding: UiAccountEncoding,
}
```

특정 계정의 데이터 변화를 실시간으로 모니터링합니다.

2. 프로그램 구독 (Program Subscription)

```
#[rpc(name = "programSubscribe")]
fn program_subscribe(
&self,
pubkey_str: String,
config: Option<RpcProgramAccountsConfig>,
) -> Result<SubscriptionId>;

pub struct ProgramSubscriptionParams {
pub pubkey: Pubkey,
pub filters: Vec<RpcFilterType>,
pub encoding: UiAccountEncoding,
pub data_slice: Option<UiDataSliceConfig>,
pub commitment: CommitmentConfig,
```

```
pub  with_context: bool,
}
```

특정 프로그램이 소유한 모든 계정의 변화를 모니터링합니다.

3. 로그 구독 (Logs Subscription)

```
#[rpc(name = "logsSubscribe")]
fn  logs_subscribe(
&self,
filter: RpcTransactionLogsFilter,
config: Option<RpcTransactionLogsConfig>,
) -> Result<SubscriptionId>;

pub  enum LogsSubscriptionKind {
All,
AllWithVotes,
Single(Pubkey),
}
```

트랜잭션 실행 로그를 실시간으로 수신합니다.

4. 서명 구독 (Signature Subscription)

```
#[rpc(name = "signatureSubscribe")]
```

```
fn signature_subscribe(
&self,
signature_str: String,
config: Option<RpcSignatureSubscribeConfig>,
) -> Result<SubscriptionId>;


pub struct SignatureSubscriptionParams {
pub signature: Signature,
pub commitment: CommitmentConfig,
pub enable_received_notification: bool,
}
```

특정 트랜잭션의 확인 상태를 추적합니다.

5. 슬롯 구독 (Slot Subscription)

```
#[rpc(name = "slotSubscribe")]
fn slot_subscribe(&self) -> Result<SubscriptionId>;


#[rpc(name = "slotsUpdatesSubscribe")]
fn slots_updates_subscribe(&self) -> Result<SubscriptionId>;
```

새로운 슬롯 생성 및 업데이트를 모니터링합니다.

6. 블록 구독 (Block Subscription)

```
#[rpc(name = "blockSubscribe")]
fn block_subscribe(
&self,
filter: RpcBlockSubscribeFilter,
config: Option<RpcBlockSubscribeConfig>,
) -> Result<SubscriptionId>;


pub enum BlockSubscriptionKind {
All,
MentionsAccountOrProgram(Pubkey),
}
```

새로운 블록 생성을 실시간으로 모니터링합니다.

WebSocket 연결 처리

연결 수락 및 핸드셰이크

```
async fn handle_connection(
socket: TcpStream,
subscription_control: SubscriptionControl,
config: PubSubConfig,
mut tripwire: Tripwire,
) -> Result<(), Error> {
```

```
// WebSocket 핸드셰이크 처리
let mut server = Server::new(socket.compat());
let request = server.receive_request().await?;
let accept = server::Response::Accept {
key: request.key(),
protocol: None,
};
server.send_response(&accept).await?;
let (mut sender, mut receiver) = server.into_builder().finish();


// 구독 관리 및 알림 처리 설정
let mut broadcast_receiver = subscription_control.broadcast_receiver();
let current_subscriptions = Arc::new(DashMap::new());
// JSON-RPC 핸들러 설정
let mut json_rpc_handler = IoHandler::new();
let rpc_impl = RpcSolPubSubImpl::new(
config,
subscription_control,
Arc::clone(&current_subscriptions),
);
json_rpc_handler.extend_with(rpc_impl.to_delegate());
}
```

이벤트 루프 처리

```
loop {
let receive_future = receiver.receive_data(&mut data);
pin!(receive_future);
loop {
select! {
// 클라이언트로부터 메시지 수신
result = &mut receive_future => match result {
Ok(_) => break,
Err(soketto::connection::Error::Closed) => return Ok(()),
Err(err) => return Err(err.into()),
},
// 브로드캐스트 알림 수신
result = broadcast_receiver.recv() => {
if let Some(json) = broadcast_handler.handle(result?)? {
sender.send_text(&*json).await?;
}
},
// 종료 신호 처리
_ = &mut tripwire => {
warn!("disconnecting websocket client: shutting down");
return Ok(())
```

```
},
}
}
// JSON-RPC 요청 처리
if let Some(response) = json_rpc_handler.handle_request(data_str).await {
sender.send_text(&response).await?;
}
}
```

알림 시스템

알림 엔트리 유형

```
pub enum NotificationEntry {
Slot(SlotInfo),
SlotUpdate(SlotUpdate),
Vote((Pubkey, VoteTransaction, Signature)),
Root(Slot),
Bank(CommitmentSlots),
Gossip(Slot),
SignaturesReceived((Slot, Vec<Signature>)),
Subscribed(SubscriptionParams, SubscriptionId),
Unsubscribed(SubscriptionParams, SubscriptionId),
```

```
}

pub struct TimestampedNotificationEntry {
pub entry: NotificationEntry,
pub queued_at: Instant,
}
```

알림 배포 메커니즘

```
impl RpcSubscriptions {
pub fn notify_subscribers(&self, commitment_slots: CommitmentSlots) {
self.enqueue_notification(NotificationEntry::Bank(commitment_slots));
}

pub fn notify_slot(&self, slot: Slot, parent: Slot, root: Slot) {
self.enqueue_notification(NotificationEntry::Slot(SlotInfo {
slot,
parent,
root,
}));
}

pub fn notify_signatures_received(&self, slot_signatures: (Slot, Vec<Signature
```

```
self.enqueue_notification(NotificationEntry::SignaturesReceived(slot_signatures)
}


fn  enqueue_notification(&self, notification_entry: NotificationEntry) {
match  self
.notification_sender
.lock()
.unwrap()
.send(notification_entry.into())
{
Ok(()) => (),
Err(SendError(_)) => {
info!("Notification thread exited");
}
}
}
}
```

구독 관리

구독 생성 및 추적

```
impl RpcSolPubSubImpl {
```

```
fn subscribe(&self, params: SubscriptionParams) -> Result<SubscriptionId> {
let token = self
.subscription_control
.subscribe(params)
.map_err(|_| Error {
code: ErrorCode::InternalError,
message: "Internal Error: Subscription refused. Node subscription limit reached"
.into(),
data: None,
})?;
let id = token.id();
self.current_subscriptions.insert(id, token);
Ok(id)
}


fn unsubscribe(&self, id: SubscriptionId) -> Result<bool> {
if self.current_subscriptions.remove(&id).is_some() {
Ok(true)
} else {
Err(Error {
code: ErrorCode::InvalidParams,
message: "Invalid subscription id.".into(),
data: None,
```

```
})
}
}
}
```

구독 상태 추적

```
pub struct SubscriptionsTracker {
subscriptions: RwLock<HashMap<SubscriptionId, SubscriptionInfo>>,
next_id: AtomicU64,
total_subscriptions: AtomicUsize,
total_subscription_count_by_type: [AtomicUsize; 9],
}

pub struct SubscriptionInfo {
pub params: SubscriptionParams,
pub sink: Arc<Mutex<Sink>>,
pub last_notified_slot: Arc<RwLock<Slot>>,
pub commitment: CommitmentConfig,
}
```

성능 최적화

브로드캐스트 채널

```
// 고성능 브로드캐스트를 위한 Tokio broadcast 채널 사용
let mut broadcast_receiver = subscription_control.broadcast_receiver();


// 알림 처리 통계
#[derive(Default)]
struct SentNotificationStats {
num_account: AtomicUsize,
num_logs: AtomicUsize,
num_program: AtomicUsize,
num_signature: AtomicUsize,
num_slot: AtomicUsize,
// ... 기타 통계 필드들
}
```

멀티스레드 처리

```
// 멀티스레드 Tokio 런타임
let runtime = tokio::runtime::Builder::new_multi_thread()
.thread_name("solRpcPubSubRt")
.worker_threads(pubsub_config.worker_threads)
.enable_all()
```

```
.build()
.expect("runtime creation failed");
```

큐 용량 관리

```
#[derive(Debug, Clone)]
pub struct PubSubConfig {
pub max_active_subscriptions: usize,
pub queue_capacity_items: usize,
pub queue_capacity_bytes: usize,
pub worker_threads: usize,
pub notification_threads: Option<NonZeroUsize>,
}

// 기본 설정값
pub const MAX_ACTIVE_SUBSCRIPTIONS: usize = 1_000_000;
pub const DEFAULT_QUEUE_CAPACITY_ITEMS: usize = 10_000_000;
pub const DEFAULT_QUEUE_CAPACITY_BYTES: usize = 256 * 1024 * 1024; // 256MB
```

알림 필터링 및 처리

Commitment Level 기반 필터링

```
fn check_commitment_and_notify<P, S, B, F, X, I>(
```

```
params: &P,
subscription: &SubscriptionInfo,
bank_forks: &Arc<RwLock<BankForks>>,
slot: Slot,
bank_method: B,
filter_results: F,
notifier: &RpcNotifier,
is_final: bool,
) -> bool
where
S: Clone + Serialize,
B: Fn(&Bank, &P) -> X,
F: Fn(X, &P, Slot, Arc<Bank>) -> (I, Slot),
{
let  bank = bank_forks.read().unwrap().get(slot);
if  let Some(bank) = bank {
let  results = bank_method(&bank, params);
let (filter_results, result_slot) = filter_results(results, params, slot, bank);
for  result  in  filter_results {
notifier.notify(
RpcResponse::from(RpcNotificationResponse {
context: RpcNotificationContext { slot },
value: result,
```

```
}),
subscription,
is_final,
);
}
}
}
```

계정 변화 감지

```
fn get_transaction_logs(
bank: &Bank,
params: &LogsSubscriptionParams,
) -> Option<Vec<TransactionLogInfo>> {
let pubkey = match &params.kind {
LogsSubscriptionKind::All | LogsSubscriptionKind::AllWithVotes => None,
LogsSubscriptionKind::Single(pubkey) => Some(pubkey),
};
let mut logs = bank.get_transaction_logs(pubkey);
if matches!(params.kind, LogsSubscriptionKind::All) {
// 투표 트랜잭션 필터링
if let Some(logs) = &mut logs {
logs.retain(|log| !log.is_vote);
```

```
}

}

logs

}
```

오류 처리 및 복구

연결 오류 처리

```
#[derive(Debug, Error)]

enum Error {

#[error("handshake error: {0}")]

Handshake(#[from] soketto::handshake::Error),

#[error("connection error: {0}")]

Connection(#[from] soketto::connection::Error),

#[error("broadcast queue error: {0}")]

Broadcast(#[from] broadcast::error::RecvError),

#[error("client has lagged behind (notification is gone)")]

NotificationIsGone,

}
```

구독 제한 및 보호

```
impl RpcSolPubSubImpl {
fn  subscribe(&self, params: SubscriptionParams) -> Result<SubscriptionId> {
let  token = self
.subscription_control
.subscribe(params)
.map_err(|_| Error {
code: ErrorCode::InternalError,
message: "Internal Error: Subscription refused. Node subscription limit reached"
.into(),
data: None,
})?;
// ...
}
}
```

모니터링 및 메트릭

알림 통계 수집

```
fn  increment_sent_notification_stats(
params: &SubscriptionParams,
notification: &RpcNotification,
```

```
stats: &Arc<SentNotificationStats>,
) {
match params {
SubscriptionParams::Account(_) => {
stats.num_account.fetch_add(1, Ordering::Relaxed);
}
SubscriptionParams::Logs(_) => {
stats.num_logs.fetch_add(1, Ordering::Relaxed);
}
// ... 기타 유형별 통계
}
stats.total_creation_to_queue_time_us.fetch_add(
notification.created_at.elapsed().as_micros() as u64,
Ordering::Relaxed,
);
stats.maybe_report();
}
```

성능 메트릭 보고

```
impl SentNotificationStats {
fn maybe_report(&self) {
if self.last_report.should_update(METRICS_REPORT_INTERVAL_MS) {
```

```
datapoint_info!(
"rpc_pubsub-sent_notifications",
("num_account", self.num_account.swap(0, Ordering::Relaxed) as i64, i64),
("num_logs", self.num_logs.swap(0, Ordering::Relaxed) as i64, i64),
("num_program", self.num_program.swap(0, Ordering::Relaxed) as i64, i64),
// ... 기타 메트릭들
);
}
}
}
```

특수 기능

테스트 지원

```
#[cfg(test)]
pub fn test_connection(
subscriptions: &Arc<RpcSubscriptions>,
) -> (RpcSolPubSubImpl, TestBroadcastReceiver) {
let current_subscriptions = Arc::new(DashMap::new());
let rpc_impl = RpcSolPubSubImpl::new(
PubSubConfig {
enable_block_subscription: true,
```

```
enable_vote_subscription: true,
queue_capacity_items: 100,
..PubSubConfig::default()
},
subscriptions.control().clone(),
Arc::clone(&current_subscriptions),
);
let broadcast_handler = BroadcastHandler::new(current_subscriptions);
let receiver = TestBroadcastReceiver {
inner: subscriptions.control().broadcast_receiver(),
handler: broadcast_handler,
};
(rpc_impl, receiver)
}
```

조건부 기능 활성화

```
fn block_subscribe(&self, filter: RpcBlockSubscribeFilter, config: Option<RpcBl
if !self.config.enable_block_subscription {
return Err(Error::new(jsonrpc_core::ErrorCode::MethodNotFound));
}
// 블록 구독 로직
}
```

```
fn  vote_subscribe(&self) -> Result<SubscriptionId> {

if !self.config.enable_vote_subscription {

return Err(Error::new(jsonrpc_core::ErrorCode::MethodNotFound));

}

// 투표 구독 로직

}
```

결론

Solana의 PubSub 시스템은 WebSocket 기반의 실시간 이벤트 알림 시스템으로, 다양한 블록체인 상태 변화를 효율적으로 모니터링할 수 있도록 설계되었습니다. 멀티스레드 처리, 브로드캐스트 채널, 그리고 세밀한 필터링 기능을 통해 높은 성능과 확장성을 제공하며, 클라이언트 애플리케이션이 블록체인과 실시간으로 상호작용할 수 있는 강력한 도구를 제공합니다.